

Panduan Simbol, Angka & Coding

PowerShell, Terminal, Python, JavaScript, JSON, Git dan lainnya

Dari nol: apa arti tiap simbol, apa maksud angka-angkanya, cara memakai terminal, dan contoh kode yang sudah jadi dan bisa langsung dicoba.

Isi panduan ini

1. Cara memakai terminal (PowerShell, CMD, Git Bash, Linux/macOS)
2. Simbol-simbol PowerShell + operator + contoh script jadi
3. Python: simbol, tipe data, operator + 5 contoh program jadi
4. Arti angka: indeks 0, biner/hex, exit code, port, versi, status HTTP
5. JSON, .env, JavaScript, HTML, Bash
6. Git (simpan & upload kode)
7. Cara baca pesan error + tabel masalah umum
8. Bedah perintah nyata dari proyek KelasTrack + cheat sheet

*Cara belajar yang disarankan: baca bagian terminal dulu, lalu **ketik sendiri** contoh kodenya (jangan cuma copy-paste) dan ubah-ubah angkanya. Dari situ paham.*

1. Terminal: tempat mengetik perintah

Terminal (atau *shell*) adalah jendela teks tempat lu mengetik perintah ke komputer, bukan klik mouse. Ada beberapa jenis, dan **simbolnya sedikit beda** antar jenis.

Terminal	Ada di	Ciri-ciri	Cara membuka
PowerShell	Windows	Tulisan awal PS C:\Users\NAMA> . Perintah gaya Verb-Noun (Get-ChildItem). Paling kuat di Windows.	Tombol Windows, ketik powershell , Enter. Atau klik kanan Start > Terminal.
CMD (Command Prompt)	Windows	Tulisan awal C:\Users\NAMA> . Lebih tua & terbatas. Perintah: dir, cd, copy.	Tombol Windows, ketik cmd , Enter.
Git Bash	Windows (setelah pasang Git)	Meniru Linux di Windows. Perintah: ls, cd, cp. Pakai / bukan \ .	Klik kanan di folder > Git Bash Here.
Terminal Linux/macOS (bash/zsh)	Linux, Mac	Awal baris user@komputer:~\$. Pemisah folder / .	Ctrl+Alt+T (Linux); Cmd+Spasi lalu ketik Terminal (Mac).
Terminal VS Code	Semua	Terminal di dalam editor, otomatis di folder proyek. Bisa pilih PowerShell/CMD/Bash.	Di VS Code tekan Ctrl+` (tanda backtick) atau menu Terminal > New Terminal.

1.1 Cara membaca prompt (tulisan di awal baris)

```
PS C:\Users\MyBook Hype\Documents> _
| |
| |
| |
| |
| +----- folder tempat lu berada sekarang
+----- "PS" = PowerShell
```

--- kursor: di sinilah lu mengetik
+---- tanda ">" artinya terminal siap menerima perintah

Setiap perintah dijalankan **di folder tempat lu berada**. Itu kenapa sering perlu **cd** (pindah folder) dulu sebelum menjalankan sesuatu. Kalau terminal bilang file tidak ditemukan, 90% penyebabnya: lu berada di folder yang salah.

1.2 Perintah dasar: PowerShell vs CMD vs Linux/Git Bash

Tujuan	PowerShell	CMD	Linux / Mac / Git Bash
Lihat isi folder	Get-ChildItem atau ls atau dir	dir	ls (ls -la lebih lengkap)
Pindah folder	Set-Location D:\Proyek atau cd D:\Proyek	cd /d D:\Proyek	cd ~/proyek
Naik satu folder	cd ..	cd ..	cd ..
Lihat lokasi sekarang	Get-Location atau pwd	cd (tanpa apa-apa)	pwd
Buat folder	New-Item -ItemType Directory baru atau mkdir baru	mkdir baru	mkdir baru
Buat file kosong	New-Item catatan.txt	type nul > catatan.txt	touch catatan.txt
Lihat isi file	Get-Content a.txt atau cat a.txt	type a.txt	cat a.txt
Salin file	Copy-Item a.txt b.txt	copy a.txt b.txt	cp a.txt b.txt

Tujuan	PowerShell	CMD	Linux / Mac / Git Bash
Pindah / ganti nama	<code>Move-Item a.txt b.txt</code>	<code>move a.txt b.txt</code>	<code>mv a.txt b.txt</code>
Hapus file	<code>Remove-Item a.txt</code>	<code>del a.txt</code>	<code>rm a.txt</code>
Hapus folder + isinya	<code>Remove-Item baru -Recurse -Force</code>	<code>rmdir /s /q baru</code>	<code>rm -rf baru</code>
Bersihkan layar	<code>Clear-Host</code> atau <code>cls</code>	<code>cls</code>	<code>clear</code>
Cetak teks	<code>Write-Host "halo"</code>	<code>echo halo</code>	<code>echo "halo"</code>
Cari teks dalam file	<code>Select-String "kata" a.txt</code>	<code>findstr kata a.txt</code>	<code>grep kata a.txt</code>
Cek program ada/tidak	<code>Get-Command python</code>	<code>where python</code>	<code>which python3</code>
Lihat variabel PATH	<code>\$env:Path</code>	<code>echo %PATH%</code>	<code>echo \$PATH</code>
Bantuan perintah	<code>Get-Help Copy-Item</code>	<code>copy /?</code>	<code>man cp</code> atau <code>cp --help</code>

1.3 Tombol keyboard yang wajib hafal di terminal

Tombol	Fungsi
Ctrl + C	Hentikan program yang sedang jalan / batalkan. Dipakai juga untuk menghentikan server. (Bukan 'copy' di terminal lama.)
Tab	Auto-lengkapi nama file/folder. Ketik <code>cd Doc</code> lalu Tab jadi <code>cd Documents</code> . Hemat waktu & mencegah salah ketik.
Panah atas / bawah	Ulangi perintah sebelumnya.
Ctrl + L (atau <code>cls</code>)	Bersihkan layar.
Klik kanan (di PowerShell lama)	Paste teks. Di terminal baru bisa Ctrl + V atau Ctrl + Shift + V.
Ctrl + R	Cari perintah lama (Linux/Mac/PowerShell terbaru).
Esc	Hapus seluruh baris yang sedang diketik (PowerShell).

1.4 Menjalankan file script

Jenis file	Cara menjalankan	Penjelasan
<code>.ps1</code> (PowerShell)	<code>.\setup.ps1</code>	Titik-garis miring = 'file di folder ini'. Wajib ada <code>.\</code> .
<code>.py</code> (Python)	<code>python main.py</code>	Di Mac/Linux sering <code>python3 main.py</code> .
<code>.js</code> (JavaScript)	<code>node contoh.js</code>	Perlu Node.js terpasang.
<code>.sh</code> (Bash)	<code>bash setup.sh</code> atau <code>./setup.sh</code>	Hanya Linux/Mac/Git Bash.
<code>.bat</code> / <code>.cmd</code>	<code>setup.bat</code>	Script CMD lama.

Masalah paling sering di Windows

Kalau `.\setup.ps1` menolak jalan dengan pesan `*running scripts is disabled on this system*`, itu aturan keamanan PowerShell (**ExecutionPolicy**).

Solusi aman, hanya untuk jendela terminal yang sedang terbuka (hilang saat ditutup):

`Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Setelah itu jalankan lagi `.\setup.ps1`. Hanya lakukan untuk script yang lu percaya.

1.5 Alur kerja khas dari nol (contoh)

```
# 1. Masuk ke folder proyek
Set-Location "C:\Users\MyBook Hype\Documents\app-clastrack-main"

# 2. Lihat isinya untuk memastikan lu di tempat yang benar
Get-ChildItem

# 3. Jalankan script persiapan
.\setup.ps1

# 4. Masuk ke backend, aktifkan environment Python, jalankan server
cd backend
.\.venv\Scripts\Activate.ps1
uvicorn main:app --port 8081

# 5. Buka browser ke http://localhost:8081
# Hentikan server dengan Ctrl + C
```

Perhatikan: tanda # di awal baris adalah komentar. Boleh ikut disalin, tidak dijalankan.

Trik hemat waktu

Ada spasi di nama folder (contoh: **MyBook Hype**)? **Apit dengan tanda kutip**: `cd "C:\Users\MyBook Hype"`. Tanpa kutip, terminal mengira itu dua kata berbeda.

Cara cepat: seret (drag) folder dari File Explorer ke jendela terminal, alamat lengkapnya otomatis terketik.

Di File Explorer, ketik **power shell** di kolom alamat folder lalu Enter: terminal langsung terbuka di folder itu.

2. Simbol-simbol PowerShell

Simbol	Nama	Fungsi	Contoh
\	backslash	Pemisah folder di Windows	C:\Users\NAMA\Documents
/	slash	Pemisah folder di web & Linux; juga pembagian	http://localhost:8081/api, 10 / 2
.	titik	Satu titik = folder ini; .. = folder di atas; juga untuk mengambil isi objek	.\setup.ps1, cd .., \$f.Name
" "	kutip dua	Teks; variabel di dalamnya diganti nilainya	"Halo \$nama" -> Halo Budi
' '	kutip satu	Teks apa adanya, variabel tidak diganti	'Halo \$nama' -> Halo \$nama
[]	kurung siku	Daftar (array), ambil item ke-sekian, atau tipe data	\$a[0], [int]"5"
{ }	kurung kurawal	Blok perintah: isi if, foreach, fungsi	if (\$x) { Write-Host "ya" }
()	kurung biasa	Mengelompokkan / kondisi; dikerjakan duluan	if (-not (Test-Path \$p))
\$	dolar	Awalan nama variabel	\$Root = "C:\\"
\$()	sub-ekspresi	Menjalankan ekspresi di dalam teks	"Total: \$(\$a.Count)"
@()	array	Membuat daftar	@("a","b","c")
@{ }	hashtable	Pasangan nama = nilai (seperti kamus)	@{ Nama="Budi"; Umur=16 }
=	sama dengan	Mengisi variabel (bukan membandingkan!)	\$x = 5
;	titik koma	Pemisah perintah dalam satu baris	cd ..; ls
:	titik dua	Penanda drive, atau pasangan di JSON; di PowerShell juga \$env:Path	C:, \$env:Path
	pipa (pipe)	Hasil perintah kiri dikirim ke kanan	ls Sort-Object Length
>	redirect	Simpan hasil ke file (menimpa)	ls > daftar.txt
>>	redirect tambah	Simpan hasil ke file (menambah di akhir)	echo baris >> log.txt
-	strip	Awalan opsi/parameter & operator	-Recurse -Force, -eq
#	pagar	Komentar satu baris	# ini catatan
<# #>	komentar blok	Komentar banyak baris	<# catatan panjang #>
`	backtick	Lanjut ke baris berikutnya, atau karakter khusus	`n = baris baru, `" = kutip literal
&	ampersand (call)	Menjalankan perintah/jalur yang disimpan dalam teks	& ".\env\Scripts\Activate.ps1"
*	bintang (wildcard)	Mewakili apa saja	*.txt = semua file berakhiran .txt
?	tanya (wildcard)	Mewakili satu karakter apa saja	file?.txt = file1.txt, fileA.txt
~	tilde	Folder 'home' pengguna	cd ~
%	persen	Sisa bagi, atau alias ForEach-Object	10 % 3 = 1
!	seru	Sama dengan -not (tidak)	if (!\$ada) { ... }

2.1 Operator perbandingan & logika PowerShell

PowerShell **tidak** memakai **==** atau **>** untuk membandingkan, melainkan kata berawalan strip:

Operator	Arti	Contoh -> hasil
<code>-eq</code>	sama dengan (equal)	<code>5 -eq 5 -> True</code>
<code>-ne</code>	tidak sama (not equal)	<code>5 -ne 3 -> True</code>
<code>-gt / -ge</code>	lebih besar / lebih besar atau sama	<code>7 -gt 5 -> True</code>
<code>-lt / -le</code>	lebih kecil / lebih kecil atau sama	<code>3 -lt 5 -> True</code>
<code>-like</code>	cocok pola wildcard	<code>"budi" -like "b*" -> True</code>
<code>-match</code>	cocok pola regex	<code>"abc123" -match "\d+" -> True</code>
<code>-contains / -in</code>	daftar memuat / ada di dalam daftar	<code>@(1,2,3) -contains 2 -> True</code>
<code>-and</code>	dan (keduanya harus benar)	<code>(\$a -gt 1) -and (\$b -lt 9)</code>
<code>-or</code>	atau (salah satu benar)	<code>\$a -or \$b</code>
<code>-not</code>	kebalikan	<code>-not \$true -> False</code>
<code>+ - * /</code>	tambah, kurang, kali, bagi	<code>2 + 3 * 4 -> 14</code>
<code>++ / --</code>	tambah 1 / kurang 1	<code>\$i++</code>
<code>+=</code>	tambah ke nilai sendiri	<code>\$x += 5</code> sama dengan <code>\$x = \$x + 5</code>

2.2 Variabel & nilai khusus PowerShell

Nama	Arti
<code>\$true / \$false</code>	Benar / salah.
<code>\$null</code>	Kosong / tidak ada nilai.
<code>\$_</code>	Item yang sedang diproses dalam loop/pipa (<code>ForEach-Object</code> , <code>Where-Object</code>).
<code>\$env:NAMA</code>	Variabel lingkungan Windows, misal <code>\$env:USERNAME</code> , <code>\$env:Path</code> , <code>\$env:USERPROFILE</code> .
<code>\$PSScriptRoot</code>	Folder tempat file script itu berada.
<code>\$?</code>	Apakah perintah terakhir berhasil (True/False).
<code>\$LASTEXITCODE</code>	Kode keluar program terakhir (0 = sukses). Lihat bab angka.
<code>\$args</code>	Argumen yang dikirim ke script/fungsi.
<code>\$PROFILE</code>	Lokasi file pengaturan PowerShell pribadi.

2.3 Pola perintah: Verb-Noun

Hampir semua perintah PowerShell (disebut **cmdlet**) berbentuk **Kata-Kerja + Kata-Benda**, jadi mudah ditebak:

Verb	Arti	Contoh cmdlet
Get	ambil / lihat	<code>Get-ChildItem</code> , <code>Get-Content</code> , <code>Get-Process</code> , <code>Get-Command</code>
Set	ubah / tetapkan	<code>Set-Location</code> , <code>Set-Content</code> , <code>Set-ExecutionPolicy</code>
New	buat baru	<code>New-Item</code> , <code>New-Object</code>
Remove	hapus	<code>Remove-Item</code> , <code>Remove-Variable</code>
Copy / Move	salin / pindah	<code>Copy-Item</code> , <code>Move-Item</code>
Test	periksa (hasil True/False)	<code>Test-Path</code> , <code>Test-Connection</code>
Write	tampilkan keluaran	<code>Write-Host</code> , <code>Write-Output</code> , <code>Write-Error</code>

Verb	Arti	Contoh cmdlet
Invoke	jalankan	<code>Invoke-WebRequest</code> , <code>Invoke-Expression</code>
Select / Where / Sort	pilih / saring / urutkan	<code>Select-Object</code> , <code>Where-Object</code> , <code>Sort-Object</code>

Parameter selalu diawali strip: `Remove-Item` `.\x` `-Recurse` `-Force`. **Tips:** ketik `Get-Help NamaPerintah -Examples` untuk melihat contoh resmi.

2.4 Contoh script PowerShell yang sudah jadi

Contoh A: `setup.ps1` - memeriksa program, membuat virtual environment Python, memasang library.

```
# setup.ps1 - menyiapkan proyek (contoh gaya KelasTrack)
$ErrorActionPreference = "Stop" # berhenti kalau ada error
$Root = Split-Path -Parent $MyInvocation.MyCommand.Path

function Has($nama) {
    return [bool](Get-Command $nama -ErrorAction SilentlyContinue)
}

Write-Host "== Cek program ==" -ForegroundColor Cyan
foreach ($p in @("python", "node", "git")) {
    if (Has $p) {
        Write-Host "[OK] $p ditemukan" -ForegroundColor Green
    } else {
        Write-Host "[GAGAL] $p belum terpasang" -ForegroundColor Red
        exit 1
    }
}

$Backend = Join-Path $Root "backend"
Set-Location $Backend

if (-not (Test-Path ".venv")) {
    Write-Host "Membuat virtual environment..."
    python -m venv .venv
}

& ".\venv\Scripts\Activate.ps1" # aktifkan venv
python -m pip install --upgrade pip
pip install -r requirements.txt

Write-Host "Selesai! Jalankan: uvicorn main:app --port 8081" -ForegroundColor Green
```

Cara kerja singkat: `function Has` membuat 'alat cek' sendiri. `foreach` mengulang daftar program. `if/else` memberi hasil OK atau GAGAL. `Join-Path` menyambung alamat folder dengan aman. `-not (Test-Path ".venv")` artinya 'kalau folder .venv BELUM ada'.

Contoh B: `rapikan_file.ps1` - mencari, mengurutkan, dan menyalin file.

```
# rapikan_file.ps1 - contoh kerja dengan file & folder
$folder = "C:\Users\NAMA\Documents\Tugas"

# 1. ambil semua file .txt, urutkan dari yang terbesar
$daftar = Get-ChildItem -Path $folder -Filter "*.txt" -Recurse |
    Sort-Object Length -Descending

# 2. tampilkan nama & ukuran (KB)
foreach ($f in $daftar) {
    $kb = [math]::Round($f.Length / 1KB, 1)
    Write-Host "$($f.Name) - $kb KB"
}

# 3. hitung ada berapa file
Write-Host "Total: $($daftar.Count) file"

# 4. buat folder backup kalau belum ada, lalu salin
$backup = Join-Path $folder "backup"
```

```
if (-not (Test-Path $backup)) { New-Item -ItemType Directory -Path $backup | Out-Null }
$daftar | Copy-Item -Destination $backup
```

Contoh C: array, hashtable, perulangan

```
# array, hashtable, perulangan
$buah = @("apel", "jeruk", "mangga")           # array
Write-Host $buah[0]                             # apel (indeks mulai 0)
Write-Host $buah.Count                          # 3

$umur = @{ Budi = 16; Sari = 15 }               # hashtable (nama = nilai)
Write-Host $umur["Budi"]                        # 16

foreach ($b in $buah) { Write-Host "Saya suka $b" }

for ($i = 1; $i -le 3; $i++) { Write-Host "Hitungan ke-$i" }

1..5 | ForEach-Object { $_ * 2 }                # 2 4 6 8 10 ($_ = item saat ini)
```

3. Python

Python adalah bahasa pemrograman yang mudah dibaca. File berakhiran `.py`. Cek terpasang atau belum dengan: `python --version`.

3.1 Simbol-simbol Python

Simbol	Nama	Fungsi	Contoh
#	pagar	Komentar (tidak dijalankan)	# hitung total
""" """	kutip tiga	Teks banyak baris / dokumentasi fungsi	"""Penjelasan fungsi"""
" " atau ' '	kutip	Teks (string). Boleh dua atau satu, hasil sama	"halo", 'halo'
=	sama dengan	Mengisi variabel	umur = 16
==	sama dengan (banding)	Membandingkan dua nilai	umur == 16 -> True
[]	kurung siku	List (daftar), atau ambil item ke-n	a = [10, 20, 30], a[0] -> 10
{ }	kurung kurawal	Dictionary (kamus) / set; juga isi f-string	{"nama": "Budi"}, f"{x}"
()	kurung biasa	Memanggil fungsi, tuple, atau mengelompokkan	print(5), (1, 2)
:	titik dua	Penutup baris if/for/def/class (wajib!), juga pasangan kunci-nilai & potongan list	if x > 3:, a[1:3]
,	koma	Pemisah item	[1, 2, 3]
.	titik	Mengambil bagian dari sebuah objek	teks.upper(), os.path
\	backslash	Karakter khusus dalam teks; lanjut baris	"baris1\nbaris2"
*	bintang	Kali; ** pangkat; *args banyak argumen	3 * 4, 2 ** 3 -> 8
//	bagi bulat	Hasil bagi tanpa koma	7 // 2 -> 3
%	persen (modulo)	Sisa bagi	7 % 2 -> 1
@	at	Dekorator: menambah fungsi tambahan pada fungsi di bawahnya	@app.get("/")
_	garis bawah	Awalan nama 'privat', atau variabel yang diabaikan	_rahasia, for _ in range(3)
__init__	dunder	Nama khusus Python (dua garis bawah di kiri-kanan)	__init__, __name__, __str__
->	panah	Petunjuk tipe nilai keluaran fungsi (opsional)	def f() -> int:
:=	walrus	Mengisi variabel sekaligus memakainya	if (n := len(a)) > 3:

Aturan terpenting Python

Python memakai **indentasi** (menjorok ke dalam, 4 spasi) untuk menandai isi blok. Bukan kurung kurawal. Satu spasi saja salah → **IndentationError**. Jangan campur Tab dan spasi. Gunakan 4 spasi konsisten. Setiap baris **if**, **for**, **while**, **def**, **class**, **try**, **else** **harus diakhiri titik dua **:**.

```
umur = 16
if umur >= 17:           # <- diakhiri titik dua
    print("Dewasa")      # <- menjorok 4 spasi = isi dari if
else:
    print("Belum")       # <- juga menjorok
print("selesai")         # <- tidak menjorok = di luar if, selalu jalan
```

3.2 Tipe data

Tipe	Arti	Contoh	Catatan
<code>int</code>	bilangan bulat	7, -3, 1000	<code>int("42")</code> mengubah teks jadi angka
<code>float</code>	bilangan desimal (pakai titik)	3.14, 0.5	Titik, bukan koma. <code>1e3</code> = 1000.0
<code>str</code>	teks	"halo"	<code>str(5)</code> mengubah angka jadi teks
<code>bool</code>	benar/salah	True, False	Huruf depan besar
<code>None</code>	kosong	None	Padanan <code>\$null</code> PowerShell
<code>list</code>	daftar berurutan, bisa diubah	[1, 2, 3]	<code>a.append(4)</code> menambah item
<code>tuple</code>	daftar yang tidak bisa diubah	(1, 2)	
<code>dict</code>	kamus: kunci -> nilai	{"a": 1, "b": 2}	<code>d["a"]</code> -> 1
<code>set</code>	kumpulan unik, tanpa urutan	{1, 2, 3}	Duplikat otomatis hilang

3.3 Operator Python

Jenis	Operator	Contoh -> hasil
Hitung	+ - * / // % **	10 / 4 -> 2.5, 10 // 4 -> 2, 10 % 4 -> 2, 2 ** 5 -> 32
Bandingkan	== != > < >= <=	5 == 5 True, 5 != 3 True
Logika	and or not	x > 1 and x < 9
Keanggotaan	in not in	"a" in "kata" -> True
Identitas	is is not	x is None
Tambah ke diri sendiri	+= -= *= /=	total += 5
Format teks (f-string)	f"... {variabel} ..."	f"Umur {umur}" -> Umur 16
Format angka dalam f-string	{x:.2f} {x:>8} {x:05d}	{3.14159:.2f} -> 3.14; {7:05d} -> 00007

3.4 Fungsi bawaan yang sering dipakai

Fungsi	Kegunaan	Contoh
<code>print()</code>	tampilkan ke layar	<code>print("halo")</code>
<code>input()</code>	minta ketikan pengguna	<code>nama = input("Nama: ")</code>
<code>len()</code>	panjang/jumlah item	<code>len([1, 2, 3])</code> -> 3
<code>range()</code>	deret angka	<code>range(3)</code> -> 0,1,2; <code>range(1, 4)</code> -> 1,2,3
<code>sum()</code> <code>min()</code> <code>max()</code>	jumlah, terkecil, terbesar	<code>sum([1, 2, 3])</code> -> 6
<code>sorted()</code>	urutkan	<code>sorted([3, 1, 2])</code> -> [1,2,3]
<code>int()</code> <code>float()</code> <code>str()</code>	ubah tipe	<code>float("2.5")</code> -> 2.5
<code>round()</code>	pembulatan	<code>round(3.14159, 2)</code> -> 3.14
<code>open()</code>	buka file	<code>open("a.txt", "r")</code>
<code>type()</code>	cek tipe	<code>type(5)</code> -> int

3.5 Virtual environment & pip (sering muncul di proyek)

venv (virtual environment, biasanya folder **.venv**) adalah 'kotak terpisah' berisi library Python milik satu proyek, supaya tidak bentrok dengan proyek lain. **pip** adalah pemasang library.

```
python -m venv .venv                # buat venv (sekali saja)

.\.venv\Scripts\Activate.ps1        # AKTIFKAN di PowerShell (Windows)
.venv\Scripts\activate.bat          # aktifkan di CMD
source .venv/bin/activate            # aktifkan di Linux / Mac / Git Bash

pip install fastapi                  # pasang satu library
pip install -r requirements.txt      # pasang semua library dari daftar
pip list                             # lihat yang terpasang
pip freeze > requirements.txt        # simpan daftar library ke file
deactivate                           # keluar dari venv
```

Setelah aktif, awal prompt berubah menjadi **(.venv) PS C:\...>**. Tanda **(.venv)** itu pertanda venv sedang aktif.

3.6 Contoh program Python yang sudah jadi

Contoh 1: hitung nilai siswa (list, dictionary, fungsi, if/elif/else, loop, f-string)

```
# contoh_nilai.py - menghitung rata-rata nilai siswa
nilai_siswa = {
    "Budi": [80, 75, 90],
    "Sari": [95, 88, 92],
    "Andi": [60, 70, 65],
}

def rata_rata(daftar):
    """Mengembalikan rata-rata dari sebuah list angka."""
    return sum(daftar) / len(daftar)

def predikat(nilai):
    if nilai >= 90:
        return "A"
    elif nilai >= 75:
        return "B"
    elif nilai >= 60:
        return "C"
    else:
        return "D"

for nama, daftar in nilai_siswa.items():
    rata = rata_rata(daftar)
    print(f"{nama:<6} rata-rata = {rata:6.2f} predikat = {predikat(rata)}")
```

Hasil yang muncul:

```
Budi rata-rata = 81.67 predikat = B
Sari rata-rata = 91.67 predikat = A
Andi rata-rata = 65.00 predikat = C
```

Arti **{nama:<6}** = rata kiri lebar 6 karakter; **{rata:6.2f}** = angka desimal, lebar 6, 2 angka di belakang koma.

Contoh 2: simpan & baca file JSON

```
# contoh_json.py - menyimpan & membaca data dari file JSON
import json
from pathlib import Path

file_data = Path("siswa.json")

siswa = [
    {"nama": "Budi", "kelas": "10A", "hadir": True},
    {"nama": "Sari", "kelas": "10B", "hadir": False},
]
```

```
# tulis ke file
with open(file_data, "w", encoding="utf-8") as f:
    json.dump(siswa, f, indent=2, ensure_ascii=False)

# baca lagi
with open(file_data, "r", encoding="utf-8") as f:
    hasil = json.load(f)

for s in hasil:
    status = "hadir" if s["hadir"] else "tidak hadir"
    print(s["nama"], "-", s["kelas"], "-", status)
```

Contoh 3: class (membuat objek)

```
# contoh_class.py - class = cetakan untuk membuat objek
class Siswa:
    def __init__(self, nama, kelas):
        self.nama = nama          # self = objek itu sendiri
        self.kelas = kelas
        self.absen = []          # daftar tanggal tidak hadir

    def catat_absen(self, tanggal):
        self.absen.append(tanggal)

    def jumlah_absen(self):
        return len(self.absen)

    def __str__(self):
        # dipanggil oleh print()
        return f"{self.nama} ({self.kelas}) - absen {self.jumlah_absen()}x"

budi = Siswa("Budi", "10A")
budi.catat_absen("2026-09-01")
budi.catat_absen("2026-09-15")
print(budi)
```

Contoh 4: menangani error dengan try/except

```
# contoh_error.py - try / except supaya program tidak langsung mati
def bagi(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        print("Tidak bisa dibagi nol!")
        return None
    finally:
        print("selesai menghitung")

print(bagi(10, 2))    # 5.0
print(bagi(10, 0))    # None
angka = int("42")    # teks -> bilangan bulat
print(angka + 8)      # 50
```

Contoh 5: API web mini dengan FastAPI (kerangka seperti backend yang punya alamat /api/settings)

```
# main.py - API sederhana dengan FastAPI (contoh; perlu: pip install fastapi uvicorn)
from fastapi import FastAPI

app = FastAPI()

settings = {"nama_sekolah": "SMA Contoh", "tahun": 2026}

@app.get("/")
def beranda():
    return {"pesan": "Server hidup"}

@app.get("/api/settings")
def baca_settings():
    return settings

@app.get("/api/siswa/{siswa_id}")
```

```
def baca_siswa(siswa_id: int):  
    return {"id": siswa_id, "nama": f"Siswa ke-{siswa_id}"}
```

Jalankan dengan: uvicorn main:app --reload --port 8081
Buka di browser: http://localhost:8081/api/settings

Di contoh 5: `@app.get("/api/settings")` artinya 'kalau ada yang membuka alamat /api/settings, jalankan fungsi di bawahnya'. `{siswa_id}` di alamat adalah bagian yang bisa berubah. `--reload` memuat ulang server otomatis saat kode diubah. `8081` adalah nomor port.

4. Arti angka-angka dalam coding

Angka di dunia coding tidak selalu berarti 'jumlah'. Ini arti yang paling sering ditemui:

4.1 Hitungan mulai dari 0 (indeks)

Hampir semua bahasa (Python, JavaScript, PowerShell, C, Java) menghitung posisi item **mulai dari 0**, bukan 1.

```
buah = ["apel", "jeruk", "mangga"]
#      0      1      2      <- nomor posisi (indeks)
#      -3     -2     -1     <- dari belakang (Python)

buah[0]    # "apel"      item pertama
buah[2]    # "mangga"    item ketiga
buah[-1]   # "mangga"    item TERAKHIR
buah[0:2]  # ["apel", "jeruk"] potong: dari 0 sampai SEBELUM 2
len(buah)  # 3           jumlah item, tapi indeks terakhir = 2
range(3)   # 0, 1, 2     3 angka, mulai 0, berhenti sebelum 3
```

4.2 Sistem bilangan: desimal, biner, heksadesimal

Sistem	Digit	Contoh	Nilai desimal	Dipakai untuk
Desimal (basis 10)	0-9	255	255	Angka sehari-hari
Biner (basis 2)	0 dan 1	0b11111111 (Python)	255	Cara kerja komputer, bit
Oktal (basis 8)	0-7	0o377 (Python)	255	Izin file Linux (chmod 755)
Heksadesimal (basis 16)	0-9, A-F	0xFF / #FF0000	255	Warna, alamat memori, MAC address

Contoh warna #2F6DB5 : 2F = merah (47), 6D = hijau (109), B5 = biru (181). Python: `int("FF", 16)` -> 255, `bin(5)` -> '0b101', `hex(255)` -> '0xff'.

Satuan data	Besar
1 bit	satu angka 0 atau 1
1 byte	8 bit (nilai 0-255)
1 KB / MB / GB	sekitar 1.000 / 1.000.000 / 1.000.000.000 byte (komputer menghitung 1024)
PowerShell 1KB, 1MB, 1GB	Konstanta ukuran bawaan, misal <code>\$f.Length / 1MB</code>

4.3 Exit code (kode keluar) - 0 berarti sukses

Setiap program yang selesai mengembalikan satu angka ke terminal:

Angka	Arti
0	Sukses, tidak ada masalah.
1 (atau angka lain bukan 0)	Ada masalah / gagal. Angkanya bisa dipilih pembuat program.
<code>exit 1</code> di script	Hentikan script dan laporkan gagal.
<code>exit 0</code>	Hentikan script dan laporkan sukses.

Cek exit code terakhir: PowerShell `$LASTEXITCODE`, CMD `echo %ERRORLEVEL%`, Bash `echo $?`.

4.4 Port - angka setelah titik dua di alamat

Komputer bisa menjalankan banyak program jaringan sekaligus. **Port** adalah nomor 'pintu' tiap program. Alamat `http://localhost:8081/api/settings` dibaca:

```

http://localhost:8081/api/settings
|           |           |           |
|           |           | +-- jalur (path): bagian mana yang diminta
|           | +----- port 8081: pintu nomor 8081
| +----- host: "localhost" = komputer lu sendiri (sama dengan 127.0.0.1)
+----- protokol: http (https = versi aman)

```

Port	Biasanya untuk
80	Web biasa (http), tidak perlu ditulis di alamat
443	Web aman (https), tidak perlu ditulis
22	SSH (akses server jarak jauh)
21	FTP
3000	Server pengembangan React / Node
5173	Vite (alat pengembangan web)
5432	PostgreSQL (database)
3306	MySQL / MariaDB (database)
27017	MongoDB (database)
8000, 8080, 8081	Server pengembangan umum (Python/FastAPI/Django/Java)

Port 0-1023 khusus sistem; untuk proyek sendiri pakai 3000-9999. Error **'port already in use'** berarti port itu sedang dipakai program lain: tutup program itu (Ctrl+C di terminalnya) atau ganti nomor port.

4.5 Alamat IP

Alamat	Arti
127.0.0.1 / localhost	Komputer lu sendiri (loopback).
0.0.0.0	'Semua alamat' - server bisa diakses dari perangkat lain di jaringan.
192.168.x.x, 10.x.x.x	Alamat jaringan lokal (rumah/kantor/sekolah), bukan internet.

4.6 Versi: 3.11.4

Format versi **MAYOR.MINOR.PATCH**. Contoh Python **3.11.4**: 3 = mayor (perubahan besar, bisa tidak kompatibel), 11 = minor (fitur baru), 4 = patch (perbaikan bug). Di **requirements.txt/package.json** ada tanda khusus:

Penulisan	Arti
fastapi==0.110.0	Harus tepat versi itu (Python/pip).
fastapi>=0.100	Versi 0.100 atau lebih baru.
"react": "^18.2.0"	Boleh naik minor/patch (18.x.x), jangan lompat ke 19 (npm).
"react": "~18.2.0"	Boleh naik patch saja (18.2.x) (npm).

4.7 Kode status HTTP (angka balasan server)

Kode	Nama	Maksud
200	OK	Berhasil.
201	Created	Berhasil membuat data baru.
204	No Content	Berhasil, tidak ada isi balasan.

Kode	Nama	Maksud
301 / 302	Redirect	Dialihkan ke alamat lain.
400	Bad Request	Permintaan salah format / data tidak lengkap.
401	Unauthorized	Belum login / kredensial salah.
403	Forbidden	Sudah login, tapi tidak punya izin.
404	Not Found	Alamat / data tidak ada. Cek ejaan URL.
422	Unprocessable	Format benar tapi isi tidak valid (sering di FastAPI).
429	Too Many Requests	Terlalu sering meminta, pelan-pelan.
500	Server Error	Bug / kesalahan di sisi server. Lihat log server.
502 / 503	Bad Gateway / Unavailable	Server mati atau sedang sibuk.

Pola: **2xx** = sukses, **3xx** = dialihkan, **4xx** = salah di pihak peminta, **5xx** = salah di pihak server.

4.8 Angka lain yang sering ditemui

Angka / bentuk	Arti	Contoh
-1	'tidak ketemu' / 'terakhir' / 'tak terbatas'	<code>buah[-1]; "abc".find("z") -> -1</code>
1 dan 0	Sering dipakai sebagai True / False	<code>DEBUG=1</code>
1e3, 2.5e-4	Notasi ilmiah: 1×10^3 dan $2,5 \times 10^{-4}$	<code>1e3 -> 1000.0</code>
1_000_000	Garis bawah pemisah ribuan agar mudah dibaca	Python: sama dengan 1000000
755, 644	Izin file Linux (pemilik/grup/lainnya: baca=4, tulis=2, jalan=1)	<code>chmod 755 setup.sh</code>
1700000000	Epoch / Unix time: detik sejak 1 Jan 1970	<code>time.time()</code>
2026-10-01	Format tanggal ISO: tahun-bulan-hari	<code>2026-10-01T08:30:00</code>
UTF-8	Standar penyimpanan huruf; utf-8 aman untuk bahasa Indonesia & emoji	<code>open(f, encoding="utf-8")</code>
\n \t \	Baris baru, tab, satu backslash dalam teks	<code>"a\nb" -> dua baris</code>
%d %s %f	Penanda format gaya lama: angka bulat, teks, desimal	<code>"Umur %d" % 16</code>
{0} {1}	Penanda urutan di <code>.format()</code> / C#	<code>"{0} dan {1}".format("a", "b")</code>

5. JSON, .env, JavaScript, HTML, Bash

5.1 JSON - format tukar data

JSON dipakai hampir di mana-mana: API, file pengaturan, `package.json`. Aturannya ketat:

```
{
  "nama": "kelastrack",
  "versi": "1.2.0",
  "aktif": true,
  "port": 8081,
  "admin": null,
  "tag": ["sekolah", "absensi"],
  "database": {
    "host": "localhost",
    "port": 5432
  }
}
```

Simbol / bentuk	Arti
{ }	Objek: kumpulan pasangan "kunci": nilai
[]	Daftar (array) nilai
"nama": "x"	Kunci selalu dalam kutip dua, titik dua memisahkan kunci dan nilai
,	Pemisah item. Tidak boleh ada koma setelah item terakhir (error umum!)
true false null	Huruf kecil semua (beda dengan Python: True, False, None)
Kutip satu ' '	Tidak sah di JSON; harus kutip dua
Komentar # atau //	Tidak diizinkan di JSON

5.2 File .env - pengaturan rahasia

```
# .env - pengaturan rahasia/lingkungan (JANGAN di-upload ke GitHub)
DATABASE_URL=postgresql://user:password@localhost:5432/kelastrack
SECRET_KEY=isi-dengan-teks-acak-panjang
PORT=8081
DEBUG=true
```

Format **NAMA=nilai**, satu per baris, tanpa spasi di sekitar **=**. Isinya sering memuat password & kunci rahasia, jadi **jangan pernah di-upload ke GitHub** (masukkan `.env` ke file `.gitignore`).

5.3 JavaScript (Node.js)

Bahasa untuk web (di browser) dan juga server (Node.js). File `.js`, dijalankan `node nama.js`.

```
// contoh.js - JavaScript dasar (jalankan: node contoh.js)
const siswa = [
  { nama: "Budi", nilai: 80 },
  { nama: "Sari", nilai: 95 },
  { nama: "Andi", nilai: 60 },
];

// arrow function: (parameter) => hasil
const lulus = siswa.filter((s) => s.nilai >= 75);

for (const s of lulus) {
  console.log(`${s.nama} lulus dengan nilai ${s.nilai}`);
}

const rata = siswa.reduce((total, s) => total + s.nilai, 0) / siswa.length;
console.log("Rata-rata:", rata.toFixed(1));
```

Simbol	Arti	Contoh
;	Akhir pernyataan (sering opsional)	<code>let x = 5;</code>
<code>const / let</code>	Buat variabel tetap / variabel bisa berubah	<code>const pi = 3.14</code>
<code>=></code>	Panah: fungsi singkat (arrow function)	<code>(a, b) => a + b</code>
<code>===</code>	Sama persis (nilai dan tipe). Gunakan ini, bukan <code>==</code>	<code>5 === "5" -> false</code>
<code>!==</code>	Tidak sama persis	<code>a !== b</code>
<code>&& !</code>	dan, atau, bukan	<code>a > 1 && a < 9</code>
`	Backtick: teks dengan <code>\${variabel}</code> di dalamnya	<code>`Halo \${nama}`</code>
<code>? :</code>	Operator ternary: kondisi ? jika benar : jika salah	<code>umur > 17 ? "dewasa" : "anak"</code>
<code>...</code>	Spread: bongkar isi array/objek	<code>[...a, 4]</code>
<code>// /* */</code>	Komentar satu baris / banyak baris	<code>// catatan</code>

npm - pemasang library JavaScript

```
node --version          # cek Node terpasang
npm install              # pasang semua library dari package.json
npm install axios        # pasang satu library
npm run dev              # jalankan skrip "dev" milik proyek (biasanya server pengembangan)
npm run build            # bangun versi final untuk dipublikasikan
npm start                # jalankan aplikasi
```

5.4 HTML + CSS + JavaScript dalam satu file

Simpan sebagai `halaman.html` lalu klik dua kali; langsung terbuka di browser.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>Halaman Pertama</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 2rem; }
    .kartu { border: 1px solid #ccc; padding: 1rem; border-radius: 8px; }
    #tombol { background: #2f6db5; color: white; padding: 8px 14px; }
  </style>
</head>
<body>
  <div class="kartu">
    <h1 id="judul">Halo Dunia</h1>
    <button id="tombol">Klik saya</button>
  </div>
  <script>
    document.getElementById("tombol").addEventListener("click", () => {
      document.getElementById("judul").textContent = "Tombol diklik!";
    });
  </script>
</body>
</html>
```

`<tag>` pembuka dan `</tag>` penutup (garis miring = penutup). `.kartu` di CSS = elemen ber-`class="kartu"`; `#tombol` = elemen ber-`id="tombol"`. `{ }` di CSS membungkus aturan gaya, dan `;` mengakhiri tiap aturan.

5.5 Bash (Linux / Mac / Git Bash)

```
#!/bin/bash
# setup.sh - versi Linux/macOS dari setup.ps1
set -e                                # berhenti kalau ada error

for p in python3 node git; do
```

```

if command -v "$p" > /dev/null; then
    echo "[OK]    $p ditemukan"
else
    echo "[GAGAL] $p belum terpasang"
    exit 1
fi
done

cd backend
[ -d .venv ] || python3 -m venv .venv      # buat venv kalau belum ada
source .venv/bin/activate                  # aktifkan venv
pip install -r requirements.txt
echo "Selesai!"

```

Bedanya dengan PowerShell: variabel dipanggil **\$nama** tetapi diisi tanpa **\$** (**nama=5**, **tanpa spasi** sekitar **=**). Kondisi pakai **[...]**, blok **if** diakhiri **fi**, perulangan **for** diakhiri **done**.

6. Git: menyimpan versi & upload kode

Git mencatat riwayat perubahan kode. GitHub adalah tempat menyimpan (online) hasil Git. Perintah dijalankan di terminal, di dalam folder proyek.

```
git --version                # cek Git terpasang

# --- mulai ---
git clone https://github.com/user/proyek.git    # unduh proyek dari GitHub
cd proyek
git init                                       # atau: jadikan folder sendiri repo Git

# --- siklus harian ---
git status                                  # lihat file apa yang berubah
git add .                                   # tandai SEMUA perubahan (titik = folder ini)
git add main.py                             # atau tandai satu file saja
git commit -m "Tambah fitur absensi"         # simpan jadi satu 'versi' + pesan
git push                                    # kirim ke GitHub
git pull                                    # ambil perubahan terbaru dari GitHub

# --- cabang (branch) ---
git branch                                  # lihat daftar cabang
git checkout -b fitur-baru                 # buat & pindah ke cabang baru
git checkout main                          # kembali ke cabang main
git merge fitur-baru                       # gabungkan fitur-baru ke cabang sekarang

# --- lihat riwayat ---
git log --oneline                         # daftar commit ringkas
git diff                                  # lihat isi perubahan
```

Istilah	Arti
repository (repo)	Folder proyek yang dilacak Git.
commit	Satu titik simpan (versi) dengan pesan.
branch	Jalur kerja terpisah, supaya eksperimen tidak merusak versi utama.
origin	Nama bawaan untuk alamat repo di GitHub.
main / master	Nama cabang utama.
HEAD	Penunjuk 'posisi saya sekarang' di riwayat.
.gitignore	File berisi daftar file/folder yang TIDAK ikut dilacak (misal <code>.venv</code> , <code>node_modules</code> , <code>.env</code>).
conflict	Dua orang mengubah baris yang sama; harus diputuskan manual mana yang dipakai.

Isi `.gitignore` contoh:

```
.venv/
node_modules/
__pycache__/
*.pyc
.env
*.log
```

7. Cara membaca pesan error

Error bukan tanda gagal, itu petunjuk. Cara membacanya:

- Baca dari **baris paling bawah**: itu jenis & penyebab error.
- Cari **nomor baris** dan **nama file** (misal **File "main.py", line 12**) - di situ letak masalahnya, atau tepat di atasnya.
- Salin kalimat error ke Google / tanya AI - hampir semua error sudah pernah dialami orang lain.
- Ubah **satu hal** per percobaan, lalu jalankan lagi.

Traceback (most recent call last):

File "main.py", line 12, in <module> <- di file & baris mana

 hasil = hitung(5)

File "main.py", line 7, in hitung

 return x / y

NameError: name 'y' is not defined <- BACA INI: variabel 'y' belum dibuat

7.1 Error umum & solusinya

Pesan error	Biasanya karena	Solusi
'python' is not recognized... / The term 'python' is not recognized	Program belum terpasang atau tidak ada di PATH	Pasang ulang; saat install Python centang Add to PATH . Tutup & buka lagi terminal.
running scripts is disabled on this system	ExecutionPolicy PowerShell	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
Cannot find path ... because it does not exist	Salah folder atau salah ketik	Get-Location & Get-ChildItem untuk cek; pakai Tab.
The term '.\setup.ps1' is not recognized	File tidak ada di folder ini	cd ke folder yang benar.
IndentationError	Spasi/jorokan Python salah	Rapikan jadi 4 spasi konsisten.
SyntaxError: invalid syntax	Salah tulis: kurang :, kurung, atau kutip	Lihat baris yang ditunjuk dan baris di atasnya.
NameError: name 'x' is not defined	Variabel belum dibuat / salah ketik / beda huruf besar-kecil	Cek ejaan; Python membedakan Nama dan nama .
TypeError: unsupported operand ... 'int' and 'str'	Mencampur angka dan teks	int(teks) atau str(angka) .
IndexError: list index out of range	Indeks melebihi jumlah item	Ingat indeks mulai 0; terakhir = len(a)-1 .
KeyError: 'nama'	Kunci dictionary tidak ada	Pakai d.get("nama") atau cek ejaan.
ModuleNotFoundError: No module named 'fastapi'	Library belum dipasang, atau venv tidak aktif	Aktifkan .venv , lalu pip install fastapi .
FileNotFoundError	Jalur file salah / kerja di folder lain	Cek folder sekarang; pakai jalur lengkap.
UnicodeDecodeError / huruf aneh	Encoding file tidak cocok	open(f, encoding="utf-8") .
Address already in use / port in use	Port sudah dipakai program lain	Tutup program lain (Ctrl+C) atau ganti port.
Connection refused / ECONNREFUSED	Server tujuan belum jalan	Jalankan server dulu; cek nomor port.
npm ERR! / command not found: npm	Node.js belum terpasang atau salah folder	Pasang Node.js; jalankan di folder yang ada package.json .

Pesan error	Biasanya karena	Solusi
Permission denied / Access is denied	Tidak punya izin	Jalankan terminal sebagai Administrator, atau cek izin file.
fatal: not a git repository	Bukan di dalam folder Git	cd ke folder proyek atau git init.
JSONDecodeError	JSON tidak valid	Cek koma berlebih, kutip satu, komentar.
Terminal diam, tidak ada kursor/prompt	Ada kutip/kurung yang belum ditutup, atau program masih berjalan	Tekan Ctrl+C .

8. Bedah perintah nyata (proyek KelasTrack)

8.1 Hapus folder .venv

```
Remove-Item ".\app-clastrack-main\backend\.venv" -Recurse -Force -ErrorAction SilentlyContinue
```

Bagian	Arti
<code>Remove-Item</code>	Perintah hapus (Verb-Noun: Remove + Item).
<code>".\app-clastrack-main\backend\.venv"</code>	Alamat target. <code>.</code> = mulai dari folder ini; tiap <code>\</code> pemisah folder; <code>.venv</code> nama folder (titik di depan = folder 'tersembunyi'). Dikutip karena aman jika ada spasi.
<code>-Recurse</code>	Hapus juga seluruh isi di dalamnya.
<code>-Force</code>	Paksa; tanpa bertanya, termasuk file tersembunyi / read-only.
<code>-ErrorAction SilentlyContinue</code>	Kalau folder tidak ada, diam saja (tidak tampil error merah).

8.2 Membuat ulang venv & memasang library

```
cd .\app-clastrack-main\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Bagian	Arti
<code>cd .\app-clastrack-main\backend</code>	Masuk ke folder backend (relatif dari folder sekarang).
<code>python -m venv .venv</code>	<code>-m</code> = jalankan modul bawaan Python bernama <code>venv</code> ; hasilnya folder <code>.venv</code> .
<code>.\.venv\Scripts\Activate.ps1</code>	Jalankan script pengaktif (<code>.</code> folder ini, <code>.venv\Scripts</code> lokasi, <code>Activate.ps1</code> filenya).
<code>pip install -r requirements.txt</code>	<code>-r</code> = baca daftar library dari file <code>requirements.txt</code> .

8.3 Menjalankan beberapa perintah sekaligus

```
cd backend; python -m venv .venv; .\.venv\Scripts\Activate.ps1; pip install -r requirements.txt
```

Titik koma `;` = 'yang ini selesai, lanjut yang itu'. Di PowerShell terbaru ada juga `&&` (lanjut hanya jika sebelumnya sukses) dan `||` (lanjut hanya jika gagal).

```
python -m venv .venv && echo "venv berhasil"      # echo hanya jika berhasil
python -m venv .venv || echo "venv gagal"         # echo hanya jika gagal
```

8.4 Menyimpan hasil & menyaring

```
Get-ChildItem -Recurse | Where-Object { $_.Length -gt 1MB } | Select-Object Name, Length
```

Dibaca dari kiri: ambil semua file termasuk subfolder (`-Recurse`) -> **pipa** | -> saring yang ukurannya (`$_ .Length`) lebih besar (`-gt`) dari 1MB -> **pipa** -> tampilkan hanya kolom Name dan Length.

```
Get-Content .\log.txt -Tail 20          # 20 baris terakhir
Select-String "ERROR" .\log.txt          # cari kata ERROR di file
Get-Process | Where-Object { $_.ProcessName -like "python*" } # proses python yang jalan
Stop-Process -Name python -Force         # matikan semua proses python
Invoke-WebRequest http://localhost:8081/api/settings          # coba panggil API
```

9. Cheat sheet satu halaman

Simbol ringkas

Simbol	Arti singkat	Simbol	Arti singkat
\	pemisah folder Windows	/	pemisah folder web/Linux; bagi
.	folder ini / ambil isi objek	..	folder di atas
" "	teks (variabel diganti di PS)	' '	teks apa adanya
[]	list/array/indeks	{ }	blok (PS/JS) / dict (Python)
()	panggil fungsi / kelompok	:	pasangan / penutup if (Python)
\$	variabel (PS/Bash)	#	komentar (PS/Python/Bash)
	pipa: kirim hasil ke kanan	;	pemisah perintah
-	awalan opsi / operator PS	&	jalankan (PS) / jalan di latar (Bash)
=	mengisi variabel	==	membandingkan (Python/JS)
> >>	simpan hasil ke file (timpa/tambah)	*	wildcard / kali
~	folder home	!	tidak (not)
%	sisa bagi	@	dekorator (Py), array/hash (PS)
=>	arrow function (JS)	->	tipe keluaran (Py) / 'menjadi' di buku ini
_	privat / diabaikan	__x__	nama khusus Python

Operator berdampingan

Fungsi	PowerShell	Python	JavaScript	Bash
sama dengan	-eq	==	===	-eq (angka) / = (teks)
tidak sama	-ne	!=	!==	-ne
lebih besar	-gt	>	>	-gt
dan	-and	and	&&	&&
atau	-or	or		
bukan	-not / !	not	!	!
komentar	#	#	//	#
kosong	\$null	None	null	(string kosong)
benar/salah	\$true \$false	True False	true false	true false
cetak	Write-Host	print()	console.log()	echo
variabel	\$x = 5	x = 5	let x = 5	x=5
if	if (c) { }	if c:	if (c) { }	if [c]; then ... fi
loop	foreach (\$i in \$a) { }	for i in a:	for (const i of a) { }	for i in a; do ... done

Lima kebiasaan yang mencegah 80% masalah

- Selalu cek dulu **lu ada di folder mana** (`pwd` / `Get-Location`).

- Pakai tombol **Tab** untuk melengkapi nama - jangan ketik panjang manual.
- Apit alamat berspasi dengan **tanda kutip**.
- Baca pesan error dari **baris paling bawah**, salin ke pencarian.
- Pakai **Ctrl+C** bila terminal menggantung; **panah atas** untuk mengulang perintah.
- Jangan upload **.env**, password, atau **.venv / node_modules** ke GitHub.

Saran terakhir

Belajar paling cepat: buka terminal, ketik tiap contoh di panduan ini sendiri, lalu **sengaja ubah** (ganti angka, hapus satu tanda kutip) dan lihat error apa yang muncul. Dari error itu lu belajar fungsi tiap simbol.